Initial Checkpoint – Operating Systems Mini Project

1. Project Details

Project Title

Distributed Load Balancer Simulator – Simulating Task Scheduling Across Multiple Nodes

Problem Statement

In modern computing systems, multiple tasks may need to be processed simultaneously using available computing resources. When tasks are distributed inefficiently, some computing nodes may become overloaded while others remain underutilized. This can lead to increased waiting time, longer completion times, and inefficient resource utilization.

The objective of this project is to develop a Distributed Load Balancer Simulator that simulates the distribution and scheduling of tasks across multiple computing nodes. The simulator will implement three scheduling algorithms: First-Come, First-Served (FCFS), Shortest Job First (SJF), and Round Robin (RR).

The system will demonstrate how different scheduling strategies affect task execution, workload distribution, and overall system performance. Performance metrics such as average waiting time, turnaround time, makespan, and node utilization will be used to compare the algorithms and identify their respective advantages and limitations.

The project will provide a practical understanding of operating system scheduling concepts through a simulated multi-node environment without requiring multiple physical computers.

Main Objectives of the Project

1. **Simulate Multiple Computing Nodes:** Create a virtual environment containing multiple computing nodes, each with its own processing capacity and assigned workload.
2. **Generate and Manage Tasks:** Simulate incoming tasks with different arrival times and execution requirements, tracking their states as pending, running, and completed.
3. **Implement Scheduling Algorithms:** Implement FCFS, SJF, and Round Robin scheduling to demonstrate different approaches to task allocation and execution.
4. **Distribute Tasks Across Nodes:** Develop a task-assignment mechanism that allocates tasks to the simulated nodes according to the selected scheduling policy.
5. **Monitor Node Workloads:** Display the workload and status of each node to identify overloaded and underutilized resources.
6. **Evaluate Scheduling Performance:** Calculate performance metrics, including average waiting time, average turnaround time, makespan, and node utilization.
7. **Compare Scheduling Algorithms:** Evaluate the three algorithms using the same set of tasks and node configurations to understand their performance under different workload conditions.
8. **Visualize the Simulation:** Provide an interface that displays task assignments, node status, scheduling activity, workload distribution, and performance comparisons.

OS Concepts and Algorithms Used

A. Operating System Concepts

1. Process and Task Management

Tasks will be represented as individual jobs with properties such as task ID, arrival time, execution time, assigned node, and execution status.

2. Process Scheduling

The simulator will demonstrate how scheduling algorithms determine the order in which tasks are selected for execution.

3. Resource Allocation

Tasks will be assigned to available computing nodes to simulate the allocation of processing resources among multiple jobs.

4. Concurrency and Parallel Execution

The simulator will model tasks executing simultaneously on different nodes, illustrating how multiple processing resources can handle work concurrently.

5. Load Balancing

The project will examine how different task-assignment strategies affect workload distribution and resource utilization across the simulated nodes.

6. Scheduling Performance Evaluation

The simulator will measure waiting time, turnaround time, makespan, and node utilization to compare the effectiveness of different scheduling strategies.

B. Algorithms Used

1. First-Come, First-Served (FCFS) Scheduling

FCFS is a non-preemptive scheduling algorithm in which tasks are selected in the order they arrive. Once a task begins execution on a node, it continues until completion.

Application in the project: FCFS will serve as a baseline for evaluating task scheduling and demonstrating the impact of arrival order on waiting time and task completion.

2. Shortest Job First (SJF) Scheduling

SJF is a non-preemptive scheduling algorithm that selects the available task with the shortest estimated execution time. Once execution begins, the selected task continues until completion.

Application in the project: SJF will be used to investigate how prioritizing shorter tasks affects average waiting time and the overall completion of the workload. Its effectiveness depends on having reliable estimates of task execution times.

3. Round Robin (RR) Scheduling

Round Robin is a preemptive scheduling algorithm that assigns each task a fixed time quantum. If a task is not completed within its allocated time, it is paused and returned to the scheduling queue so that other tasks can receive execution time.

Application in the project: Round Robin will demonstrate time-sliced scheduling and provide a basis for comparing cyclic scheduling with arrival-order and execution-time-based scheduling strategies.

C. Performance Metrics

The following metrics will be used to evaluate and compare the algorithms:

1. Average Waiting Time
The average time tasks spend waiting before execution begins, including any additional waiting between execution periods for preemptive scheduling.

2. Average Turnaround Time
The average time taken by tasks from their arrival until their completion. Turnaround Time = Completion Time − Arrival Time.

3. Makespan
The total elapsed time from the beginning of the simulation until all tasks have completed execution.

4. Node Utilization
The percentage of the simulation period during which a node is actively processing tasks.

5. Workload Distribution
The distribution of assigned work across the available nodes, used to identify imbalances in resource allocation.

Expected Outcome

The expected outcome is a functional Distributed Load Balancer Simulator that implements FCFS, SJF, and Round Robin scheduling in a multi-node simulation environment. The system will visualize task execution and node workloads while providing performance measurements for comparing the three algorithms.

The project is expected to demonstrate the trade-offs between arrival-order scheduling, shortest-task scheduling, and time-sliced scheduling. It will also provide practical insight into process scheduling, resource allocation, concurrency, and workload distribution as covered in the Operating Systems syllabus.

2. Initial Implementation

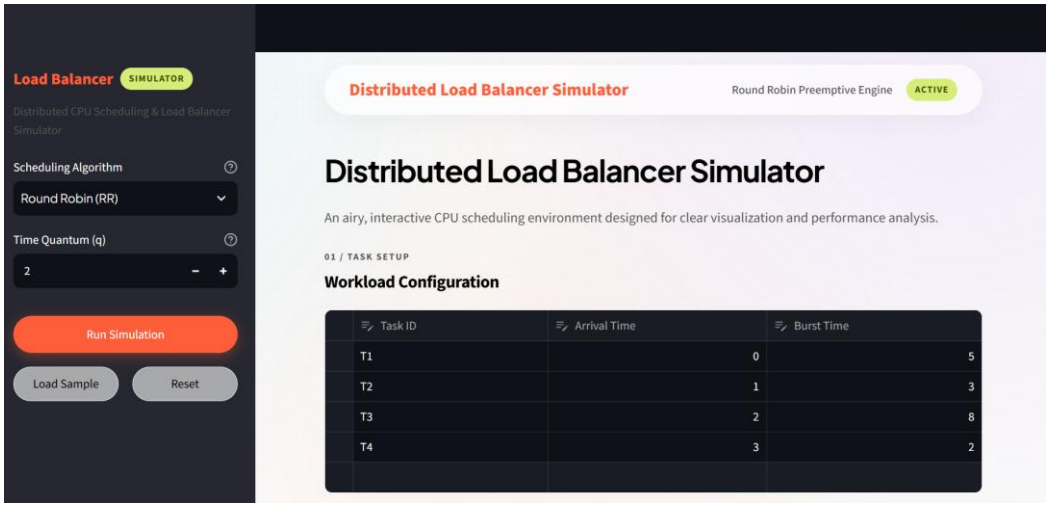
Source code of the work completed so far

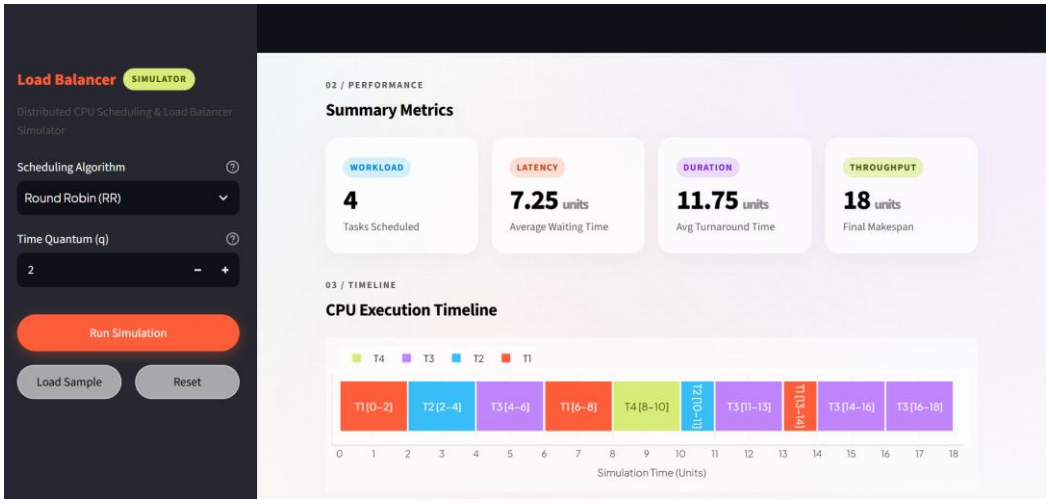
- GitHub repo: https://github.com/gangalsagar/Distributed-Load-Balancer-Simulator.git
- The code is also attached with the zip file.

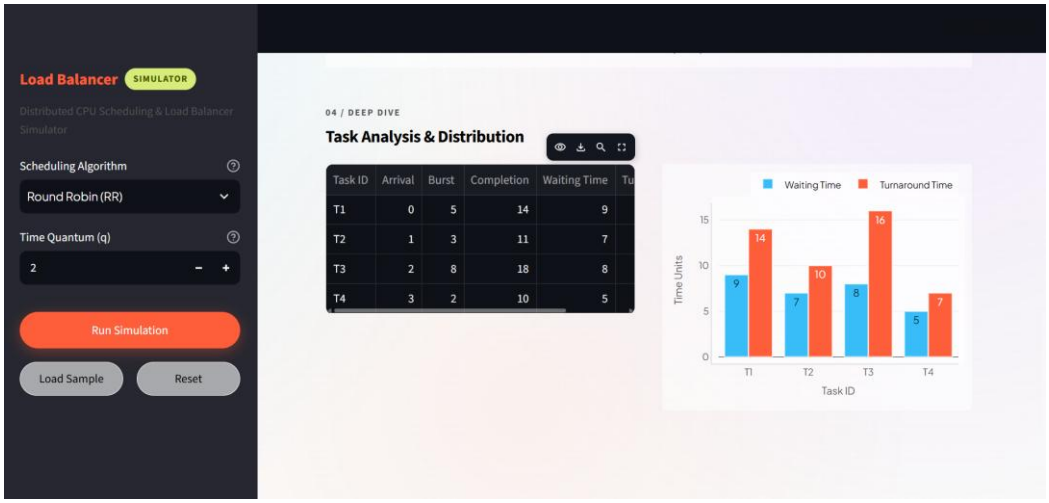
Features added so far

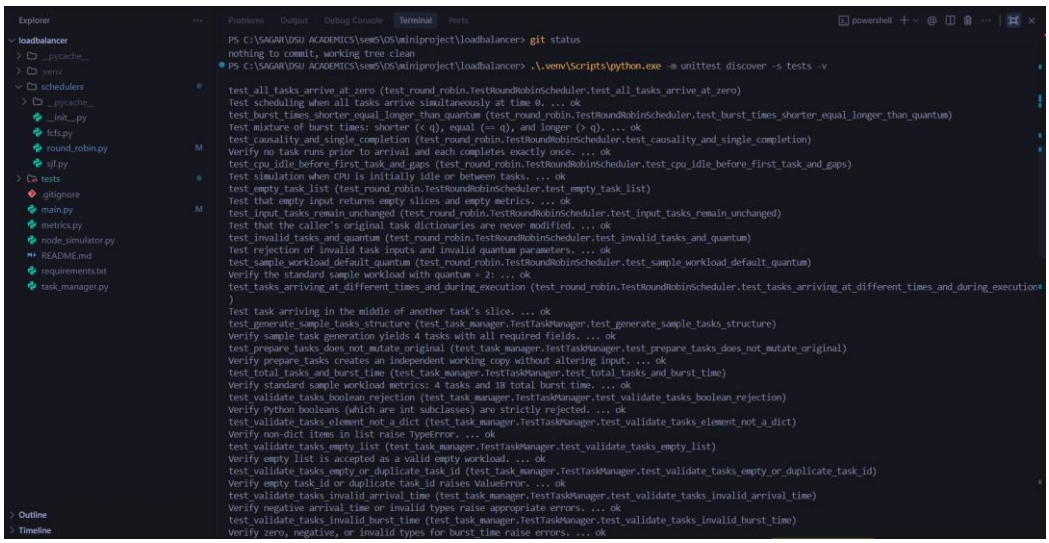
- Done with the complete implementation of the first algorithm, that is Round Robin.

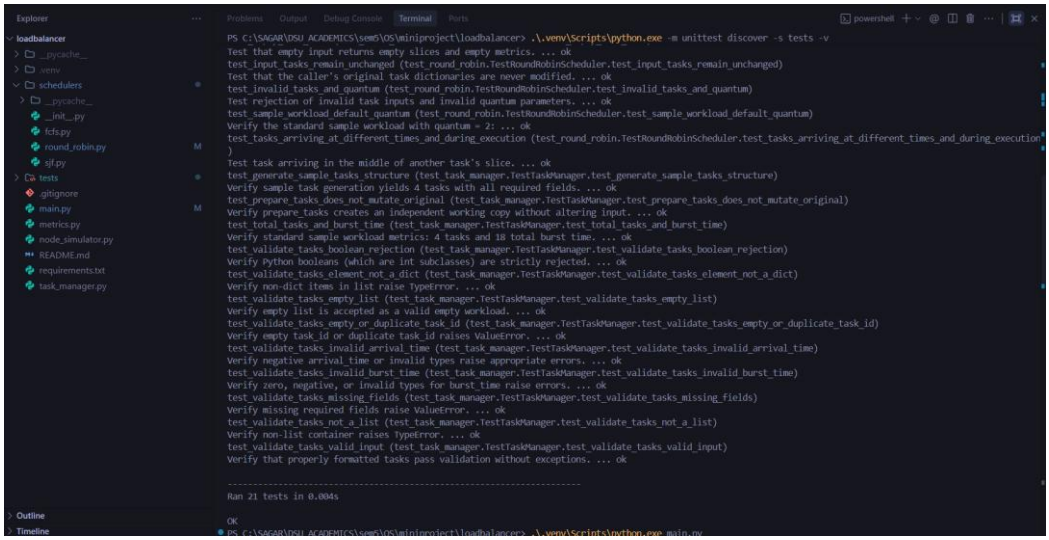
Screenshots showing the output

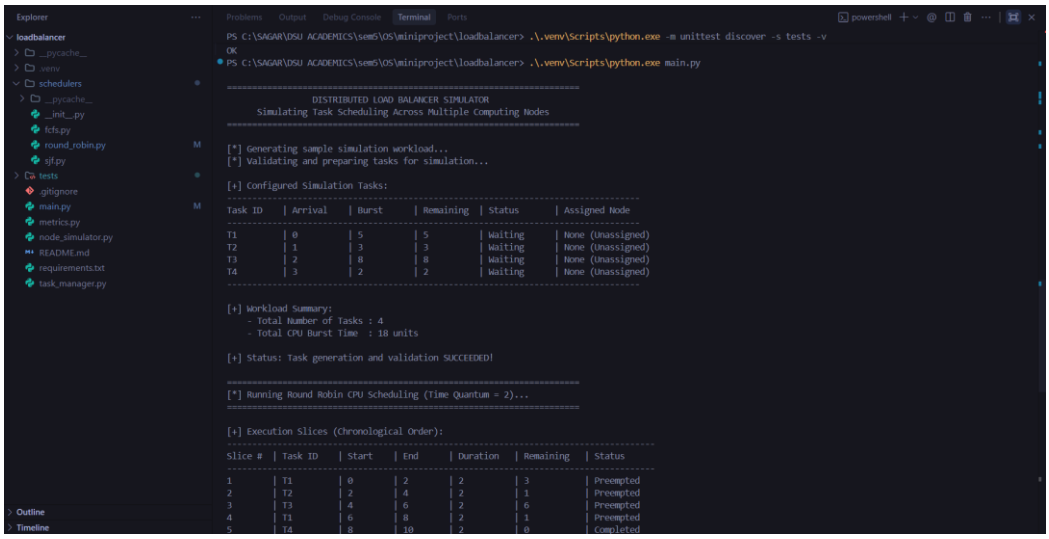


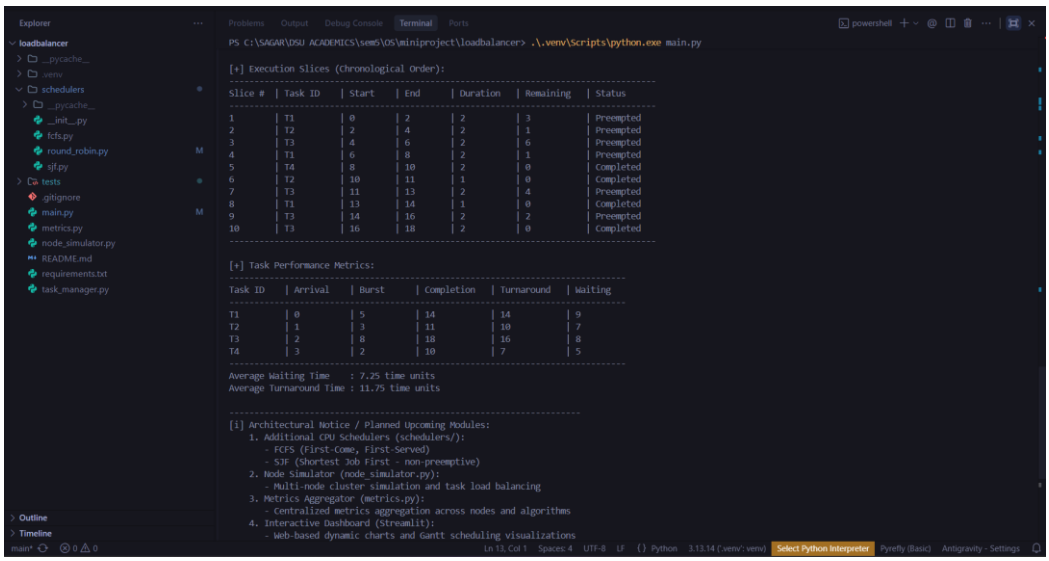





Explorer
loadbalancer
__pycache__
.venv
schedulers
__pycache__
__init__.py
fcfs.py
round_robin.py
sjf.py
tests
.gitignore
main.py
metrics.py
node_simulator.py
README.md
requirements.txt
task_manager.py
Outline
Timeline
Problems
Output
Debug Console
Terminal
Ports
powershell
PS C:\SAGAR\DSU ACADEMICS\sem5\OS\miniproject\loadbalancer> git status
nothing to commit, working tree clean
PS C:\SAGAR\DSU ACADEMICS\sem5\OS\miniproject\loadbalancer> .\.venv\Scripts\python.exe -m unittest discover -s tests -v
test_all_tasks_arrive_at_zero (test_round_robin.TestRoundRobinScheduler.test_all_tasks_arrive_at_zero)
Test scheduling when all tasks arrive simultaneously at time 0. ... ok
test_burst_times_shorter_equal_longer_than_quantum (test_round_robin.TestRoundRobinScheduler.test_burst_times_shorter_equal_longer_than_quantum)
Test mixture of burst times: shorter (< q), equal (== q), and longer (> q). ... ok
test_causality_and_single_completion (test_round_robin.TestRoundRobinScheduler.test_causality_and_single_completion)
Verify no task runs prior to arrival and each completes exactly once. ... ok
test_cpu_idle_before_first_task_and_gaps (test_round_robin.TestRoundRobinScheduler.test_cpu_idle_before_first_task_and_gaps)
Test simulation when CPU is initially idle or between tasks. ... ok
test_empty_task_list (test_round_robin.TestRoundRobinScheduler.test_empty_task_list)
Test that empty input returns empty slices and empty metrics. ... ok
test_input_tasks_remain_unchanged (test_round_robin.TestRoundRobinScheduler.test_input_tasks_remain_unchanged)
Test that the caller's original task dictionaries are never modified. ... ok
test_invalid_tasks_and_quantum (test_round_robin.TestRoundRobinScheduler.test_invalid_tasks_and_quantum)
Test rejection of invalid task inputs and invalid quantum parameters. ... ok
test_sample_workload_default_quantum (test_round_robin.TestRoundRobinScheduler.test_sample_workload_default_quantum)
Verify the standard sample workload with quantum = 2: ... ok
test_tasks_arriving_at_different_times_and_during_execution (test_round_robin.TestRoundRobinScheduler.test_tasks_arriving_at_different_times_and_during_execution)
Test task arriving in the middle of another task's slice. ... ok
test_generate_sample_tasks_structure (test_task_manager.TestTaskManager.test_generate_sample_tasks_structure)
Verify sample task generation yields 4 tasks with all required fields. ... ok
test_prepare_tasks_does_not_mutate_original (test_task_manager.TestTaskManager.test_prepare_tasks_does_not_mutate_original)
Verify prepare_tasks creates an independent working copy without altering input. ... ok
test_total_tasks_and_burst_time (test_task_manager.TestTaskManager.test_total_tasks_and_burst_time)
Verify standard sample workload metrics: 4 tasks and 18 total burst time. ... ok
test_validate_tasks_boolean_rejection (test_task_manager.TestTaskManager.test_validate_tasks_boolean_rejection)
Verify Python booleans (which are int subclasses) are strictly rejected. ... ok
test_validate_tasks_element_not_a_dict (test_task_manager.TestTaskManager.test_validate_tasks_element_not_a_dict)
Verify non-dict items in list raise TypeError. ... ok
test_validate_tasks_empty_list (test_task_manager.TestTaskManager.test_validate_tasks_empty_list)
Verify empty list is accepted as a valid empty workload. ... ok
test_validate_tasks_empty_or_duplicate_task_id (test_task_manager.TestTaskManager.test_validate_tasks_empty_or_duplicate_task_id)
Verify empty task_id or duplicate task_id raises ValueError. ... ok
test_validate_tasks_invalid_arrival_time (test_task_manager.TestTaskManager.test_validate_tasks_invalid_arrival_time)
Verify negative arrival_time or invalid types raise appropriate errors. ... ok
test_validate_tasks_invalid_burst_time (test_task_manager.TestTaskManager.test_validate_tasks_invalid_burst_time)
Verify zero, negative, or invalid types for burst_time raise errors. ... ok


Explorer
loadbalancer
schedulers
tests
main.py
Outline
Timeline
Problems
Output
Debug Console
Terminal
Ports
powershell
PS C:\SAGAR\DSU ACADEMICS\sem5\OS\miniproject\loadbalancer> .\.venv\Scripts\python.exe -m unittest discover -s tests -v
test_tasks_arriving_at_different_times_and_during_execution (test_round_robin.TestRoundRobinScheduler.test_tasks_arriving_at_different_times_and_during_execution)
Test task arriving in the middle of another task's slice. ... ok
test_generate_sample_tasks_structure (test_task_manager.TestTaskManager.test_generate_sample_tasks_structure)
Verify sample task generation yields 4 tasks with all required fields. ... ok
test_prepare_tasks_does_not_mutate_original (test_task_manager.TestTaskManager.test_prepare_tasks_does_not_mutate_original)
Verify prepare_tasks creates an independent working copy without altering input. ... ok
test_total_tasks_and_burst_time (test_task_manager.TestTaskManager.test_total_tasks_and_burst_time)
Verify standard sample workload metrics: 4 tasks and 18 total burst time. ... ok
test_validate_tasks_boolean_rejection (test_task_manager.TestTaskManager.test_validate_tasks_boolean_rejection)
Verify Python booleans (which are int subclasses) are strictly rejected. ... ok
test_validate_tasks_element_not_a_dict (test_task_manager.TestTaskManager.test_validate_tasks_element_not_a_dict)
Verify non-dict items in list raise TypeError. ... ok
test_validate_tasks_empty_list (test_task_manager.TestTaskManager.test_validate_tasks_empty_list)
Verify empty list is accepted as a valid empty workload. ... ok
test_validate_tasks_empty_or_duplicate_task_id (test_task_manager.TestTaskManager.test_validate_tasks_empty_or_duplicate_task_id)
Verify empty task_id or duplicate task_id raises ValueError. ... ok
test_validate_tasks_invalid_arrival_time (test_task_manager.TestTaskManager.test_validate_tasks_invalid_arrival_time)
Verify negative arrival_time or invalid types raise appropriate errors. ... ok
test_validate_tasks_invalid_burst_time (test_task_manager.TestTaskManager.test_validate_tasks_invalid_burst_time)
Verify zero, negative, or invalid types for burst_time raise errors. ... ok
test_validate_tasks_missing_fields (test_task_manager.TestTaskManager.test_validate_tasks_missing_fields)
Verify missing required fields raise ValueError. ... ok
test_validate_tasks_not_a_list (test_task_manager.TestTaskManager.test_validate_tasks_not_a_list)
Verify non-list container raises TypeError. ... ok
test_validate_tasks_valid_input (test_task_manager.TestTaskManager.test_validate_tasks_valid_input)
Verify that properly formatted tasks pass validation without exceptions. ... ok
Ran 21 tests in 0.004s
OK
PS C:\SAGAR\DSU ACADEMICS\sem5\OS\miniproject\loadbalancer> .\.venv\Scripts\python.exe main.py

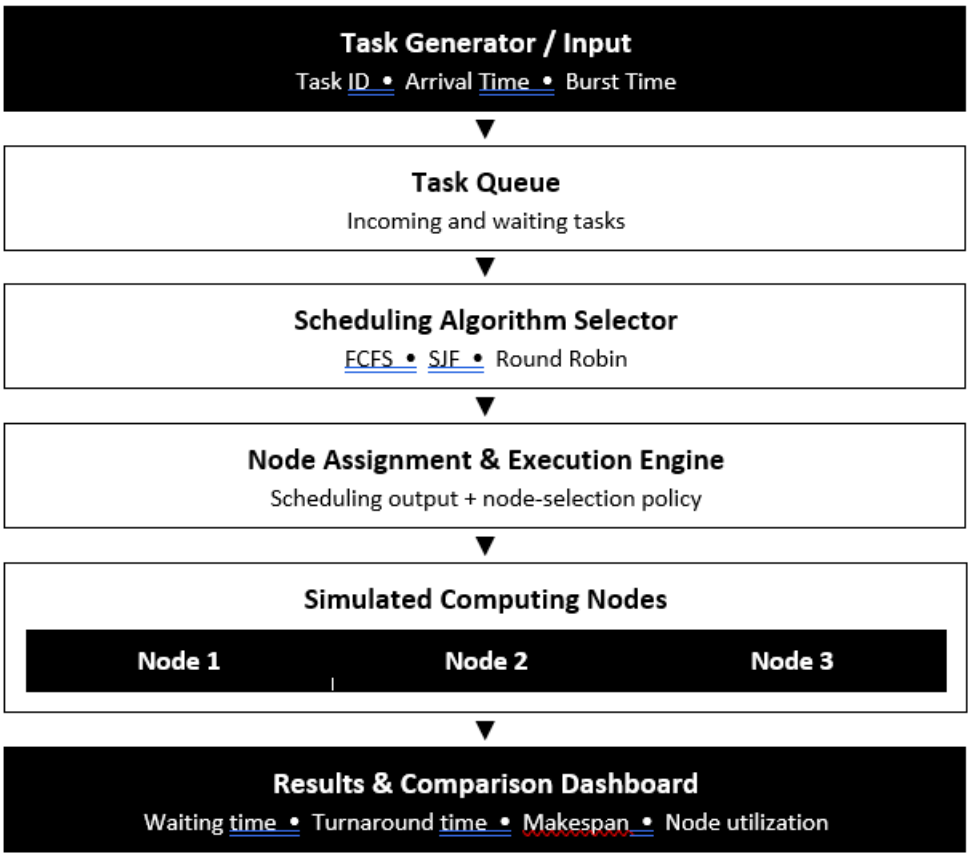

Explorer
loadbalancer
schedulers
tests
main.py
Outline
Timeline
Problems
Output
Debug Console
Terminal
Ports
powershell
PS C:\SAGAR\DSU ACADEMICS\sem5\OS\miniproject\loadbalancer> .\.venv\Scripts\python.exe -m unittest discover -s tests -v
OK
PS C:\SAGAR\DSU ACADEMICS\sem5\OS\miniproject\loadbalancer> .\.venv\Scripts\python.exe main.py
DISTRIBUTED LOAD BALANCER SIMULATOR
Simulating Task Scheduling Across Multiple Computing Nodes
[*] Generating sample simulation workload...
[*] Validating and preparing tasks for simulation...
[+] Configured Simulation Tasks:
Task ID | Arrival | Burst | Remaining | Status | Assigned Node
T1 | 0 | 5 | 5 | Waiting | None (Unassigned)
T2 | 1 | 3 | 3 | Waiting | None (Unassigned)
T3 | 2 | 8 | 8 | Waiting | None (Unassigned)
T4 | 3 | 2 | 2 | Waiting | None (Unassigned)
[+] Workload Summary:
- Total Number of Tasks : 4
- Total CPU Burst Time : 18 units
[+] Status: Task generation and validation SUCCEEDED!
[*] Running Round Robin CPU Scheduling (Time Quantum = 2)...
[+] Execution Slices (Chronological Order):
Slice # | Task ID | Start | End | Duration | Remaining | Status
1 | T1 | 0 | 2 | 2 | 3 | Preempted
2 | T2 | 2 | 4 | 2 | 1 | Preempted
3 | T3 | 4 | 6 | 2 | 6 | Preempted
4 | T1 | 6 | 8 | 2 | 1 | Preempted
5 | T4 | 8 | 10 | 2 | 0 | Completed



3. Design and Work Plan

System Block Diagram

The system accepts or generates tasks, places them in a task queue, applies the selected scheduling algorithm, assigns tasks to simulated nodes, and displays execution results and performance metrics. The stages, in order:



Design note: Scheduling decides which task runs and for how long; node assignment decides which simulated node receives the task. These are separate responsibilities and should use agreed interfaces.

Project Flowchart

▶	**Start**
1	Generate or enter tasks
2	Create simulated nodes
3	Select scheduling algorithm *FCFS, SJF, or RR*
4	Arrange tasks in the scheduling queue
5	Select the next task / time slice
6	Assign the task to a node
7	Simulate task execution and record events
8	**Decision: are all tasks completed?** NO → return to step 5 (scheduling queue) YES → continue to step 9
9	Calculate performance metrics
10	Display results and compare algorithms
■	**End**

Division of Work Among Four Team Members

MEMBER	RESPONSIBILITY	DELIVERABLES
MEMBER 1 **Task Management**	Build task input/generation, validate task fields, maintain the task queue, define the shared task data structure.	Standardized task list · input validation · sample workloads
MEMBER 2 **Scheduling Algorithms**	Implement and test FCFS, non-preemptive SJF, and Round Robin with a configurable time quantum.	Algorithm functions that accept the shared task format and return schedule / time-slice information
MEMBER 3 **Node Simulation & Metrics**	Create simulated nodes, implement the node-assignment policy, simulate execution, record events, calculate metrics.	Node workload records · execution timeline · waiting time, turnaround time, makespan, utilization
MEMBER 4 **Dashboard & Integration**	Build the interface, algorithm selector, visualizations and comparison view; integrate modules; run end-to-end tests.	Working dashboard · charts/tables · integrated application · final test run

Integration Order

1. Member 1 provides the agreed task data structure and a consistent set of test workloads.

2. Member 2 implements each scheduler using that shared format and documents how schedule decisions are returned.
3. Member 3 consumes the scheduling output, applies the node-assignment policy, and produces execution records and metrics.
4. Member 4 connects the modules to the dashboard and verifies that each algorithm can be run against the same workload.

Important: Agree on the task fields, schedule output format, and node-assignment rule before parallel development. This prevents incompatible modules and makes algorithm comparisons fair.

Methodology

PHASE 1	**Requirement Analysis** Define the simulator inputs (task ID, arrival time, burst time), number of nodes, scheduling algorithms, configurable Round Robin time quantum, outputs, and evaluation metrics. Set the scope as a simulation; real distributed machines are not required for the initial version.
PHASE 2	**System Design** Design the modular architecture: task generator and queue, scheduler, node-assignment/execution engine, metrics calculator, and dashboard. Define shared data structures and interfaces before implementation.
PHASE 3	**Algorithm Development** Implement FCFS (arrival order), non-preemptive SJF (shortest available job), and Round Robin (each ready task runs for at most a fixed quantum; unfinished tasks return to the queue). Test each algorithm independently on small, hand-checkable workloads.
PHASE 4	**Node Simulation & Metrics** Initialize the simulated nodes and apply a consistent node-assignment policy. Run each task per the scheduler output, advance the simulation clock, and record start, finish, preemption, and node-activity events. Calculate waiting time, turnaround time, makespan, and node utilization.
PHASE 5	**Dashboard & Integration** Build a simple dashboard to enter or generate tasks, choose the algorithm, set the number of nodes and Round Robin quantum, run/reset simulations, inspect task status and node workloads, and compare results. Integrate all modules through the agreed interfaces.
PHASE 6	**Testing & Evaluation** Test equal and mixed burst times, different arrival times, a single task, empty input, different quantums, and cases where some nodes stay idle. Verify every task completes once, none runs before arrival, remaining time never goes negative, and execution intervals are consistent. Run all algorithms on the same task set and node configuration for a fair comparison.

Performance Metrics

Metric	Meaning / Calculation
Waiting time	Total time a task spends waiting in the ready queue. For preemptive Round Robin, include all queue-wait intervals.
Turnaround time	Completion time − arrival time.
Makespan	Latest task completion time − simulation start time.
Node utilization	(Node busy time ÷ observation period) × 100%.

Suggested Tools

Python for simulation logic; Streamlit for the dashboard; Pandas for tabular results; Plotly for charts; Git/GitHub for team collaboration. A database or cloud deployment is not necessary for the first version.

Expected Outcome

A functional simulator that demonstrates task scheduling across multiple simulated nodes, visualizes node workloads, and compares FCFS, SJF, and Round Robin using consistent workloads and measurable performance metrics.